DAYANANDA SAGAR COLLEGE OF ARTS SCIENCE AND COMMERCE

KUMARSWAMY LAYOUT, BENGALURU-560 111

DEPARTMENT OF COMPUTER APPLICATIONS (MCA)

PROJECT ON,
BINARY CALCULATOR USING LOGIC GATES

UNDER THE GUIDANCE OF:
MRS. AKSHATA DEEPAK,
ASST PROFESSOR,
DEPARTMENT OF COMPUTER APPLICATIONS

SUBMITTED BY:
HEMANTH KUMAR P U (P03CJ25S126023)
JAIRAJ A (P03CJ25S126026)
KOUSHIK M G (P03CJ25S126029)

CONTENT

Sl.no	Topic	Page no
1.	Abstract	1
2.	Introduction	2
3.	Problem Statement	3
4.	Hardware and software requirements	4
5.	Circuit Design and Implementation	5-7
6.	Output	8
7.	Conclusion	9
8.	References	10

INTRODUCTION

The Binary Calculator Using Logic Gates is a digital logic simulation project that implements a 4-bit Ripple Carry Adder-Subtractor. This project demonstrates how fundamental logic gates—specifically XOR, AND, and OR gates—can be systematically combined to perform arithmetic operations on binary numbers.

A key feature of this system is its ability to perform both addition and subtraction using the same hardware circuit, which improves efficiency and reduces complexity. This is achieved through a single control input known as the Mode bit. When the Mode is set to 0, the circuit performs binary addition, and when it is set to 1, it performs subtraction using the concept of 2's complement.

The adder-subtractor is constructed using a series of Full Adders connected in a ripple carry configuration, where the carry output of one stage becomes the carry input of the next stage. For subtraction, the second operand is modified using XOR gates based on the Mode bit, effectively generating the 2's complement required for subtraction.

Another important feature of this project is its dual interpretation of results. The output binary result is interpreted in two different ways:

- As an Unsigned number, ranging from 0 to 15
- As a Signed 2's Complement number, ranging from −8 to +7

This dual representation helps in understanding how binary arithmetic is handled in digital systems at both hardware and software levels.

Overall, this project provides a practical understanding of how basic logic gates can be used to design complex arithmetic circuits, forming the foundation of modern processors and digital systems.

OBJECTIVE

The primary objectives of this project are as follows:

• Circuit Simulation:
To design and simulate a functional 4-bit adder-subtractor circuit using fundamental logic gates such as XOR, AND, and OR, thereby demonstrating the practical implementation of digital arithmetic circuits.

• Mode Control Mechanism:
To implement a control system using a Mode bit, which enables conditional bit inversion through XOR gates, allowing the circuit to seamlessly switch between addition (Mode = 0) and subtraction (Mode = 1) operations.

• Arithmetic Logic Implementation:
To demonstrate the use of 2's Complement arithmetic for performing subtraction by reusing addition hardware, highlighting efficiency in digital circuit design.

• Error Detection and Analysis:
To incorporate mechanisms for detecting computational errors by analyzing Carry Out (in unsigned arithmetic) and Overflow conditions (in signed arithmetic), ensuring accurate interpretation of results.

• Ripple Carry Visualization:
To illustrate the step-by-step propagation of carry across multiple full adder stages, helping in understanding the working of a ripple carry system at the bit level.

BLOCK DIAGRAM

1. The Input and Control Stage

This is the start of your system, representing the interface where the user interacts with the calculator. You should draw separate blocks for:

- Input Bus A (4-Bits): This represents the four bit-switch elements on your breadboard for the first operand.
- Input Bus B (4-Bits): This represents the four bit-switch elements for the second operand.
- Mode Selector (1-Bit): This block represents the tactile toggle switch. A single line from this block will act as a control signal.

2. The Conditional Inverter Unit

This functional unit is responsible for preparing input B for subtraction. It is controlled by the single Mode signal. You should create a 4-bit XOR logic block.

Draw four 2-Input XOR Gates (or one 4-bit XOR block). Each gate receives one bit from Input Bus B and the single control signal from the Mode Selector. This unit is the "heart" of the 2's complement process, and its output is a modified version of Input B.

3. The 4-Bit Arithmetic Unit (The Adder)

This block is the primary computation engine. It consists of the four full adder stages connected together.

Draw four boxes labeled Full Adder 0 (LSB) through Full Adder 3 (MSB).

- Inputs: Each full adder must show three input lines: one bit from Input Bus A, one corresponding bit from the Conditional Inverter Unit, and a Carry In (C_{in}).
- Carries: Draw an arrow for the initial Carry In (C_0) coming directly from the Mode Selector. Then, draw arrows showing the Carry Out of each full adder linking to the Carry In of the next bit.
- Outputs: Each full adder stage will produce two output lines: one bit for the Sum (S) and one for the Carry Out (C_{out}).

4. The Output and Analysis Stage

This final stage is where the raw data from the arithmetic unit is stored and interpreted for the user.

- Result Register (4-Bits): Connect the four Sum (S) outputs from the arithmetic unit into this block. This represents the LED display.
- Carry Out (1-Bit): Connect the final Carry Out (C_4) from Full Adder 3 to this dedicated block.
- Overflow Detection (V): This is a critical analysis component. Draw a single XOR gate that takes two specific inputs: the Carry In into bit 3 and the final Carry Out (C_4). Its output is the V (Overflow) flag.
- Unsigned and Signed Interpretation: Draw two distinct blocks connected to the Result Register. These blocks represent the visual display panels. They interpret the exact same four bits differently based on standard binary representation. The Overflow Detector connects to the Signed Interpretation block to trigger error warnings.

SIMULATOR DETAILS & IMPLEMENTATION FILES

1. SYSTEM IMPLEMENTATION

The project architecture is realized through two distinct but logically identical platforms: an Interactive Web-Based Simulator and a MATLAB-based Command-Line Calculator. Both versions utilize identical gate-level logic to ensure mathematical consistency across different environments.

1.1 Web-Based Interactive Simulator (index.html)

The web version focuses on visual interaction and hardware emulation. It provides a tactile experience that mimics a physical laboratory setup.

- Tactile Control Interface: Implements virtual DIP switches for Input A and Input B, allowing users to toggle bits manually.
- Dynamic Visual Feedback: Features a "Glow" effect on LED outputs to indicate a logic '1' state and uses real-time color transitions on the circuit schematic.
- Step-by-Step Logic Trace: A unique debugging feature that breaks down the calculation into individual Full Adder stages, explicitly showing A_i, B'_i, and Carry_{in} for every bit position.
- SVG Schematic Engine: Programmatically generates a logic diagram that highlights the active signal paths based on the current user input.

1.2 MATLAB Implementation (binary_calculator.m)

The binary_calculator.m file serves as the algorithmic backbone of the project, providing a robust command-line interface for the logic circuit. It is optimized for algorithmic verification.

- Input Validation Loop: The script includes a robust validation mechanism that forces the user to enter exactly four bits (0s and 1s only) in an array format (e.g., [1 0 1 0]).
- Gate-Level Modeling: Unlike simple decimal arithmetic, the script strictly simulates hardware behavior through:
  - Conditional Inversion: Uses the xor(B, mode) function to bitwise-invert Input B when the mode is set to Subtraction (Mode = 1).
  - Loop-Based Ripple Carry: A for loop iterates from the LSB (i=4) to the MSB (i=1), calculating the sum and carry for each stage using Boolean logic:

    S(i) = A(i) \oplus B_{mod}(i) \oplus Carry

- Advanced Diagnostics:
  - Overflow Logic: It specifically captures C_3(carry into the MSB) and C_4 (carry out) to calculate the overflow flag: overflow = xor(C3, C4).
  - Signed Conversion: It programmatically handles 2's complement conversion for display using the architectural formula: if S(1) == 1, sR = uR - 16; end.
- Formatted Reporting: The script utilizes fprintf to generate a structured text report, detailing the binary result, unsigned decimals, signed decimals, and explicit warnings if an overflow is detected.

1.3 Implementation Files Summary

File Name	Primary Language	Role in Project
index.html	HTML5 / CSS3 / JS	UI/UX, Hardware Emulation, & Visual Simulation
calculator.m	MATLAB	Algorithmic Verification & Mathematical Analysis
README.md	Markdown	Technical Documentation & Setup Instructions

IMPLEMENTATION

The implementation utilizes combinational logic to perform arithmetic. The primary architectural goal is to use a single set of Full Adders to handle both addition and subtraction.

A. Full Adder Logic

The Full Adder is the fundamental building block of the 4-bit parallel circuit. It processes three inputs: two bits to be added (A and B') and a carry-in (C_{in}) from the previous stage.

Full Adder Truth Table:

A	B′	Cin	Sum (S)	Carry Out (Cout)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1

A	B′	Cin	Sum (S)	Carry Out (Cout)
1	1	1	1	1

Boolean Equations:

- Sum (S):

  S = A \oplus B' \oplus C_{in}

- Carry Out (C_{out}):

  C_{out} = (A \cdot B') + (B' \cdot C_{in}) + (A \cdot C_{in})

B. Subtractor Logic

Subtraction is implemented as the addition of the 2's complement of the second operand.

1. XOR Conditional Inversion: Input B is processed as B' = B \oplus Mode.
   - If Mode = 0: B' = B (Arithmetic Addition).
   - If Mode = 1: B' = \overline{B} (1's complement generation).
2. Initial Carry (C_0): The Mode bit is fed directly into the first Carry-In (C{in0}). When Mode = 1, this adds 1 to the 1's complement, successfully completing the 2's complement transformation (A + \overline{B} + 1 = A - B).

Direct Full Subtractor Logic (Reference for comparison):

While our circuit uses adders, the direct logic for subtraction ($A - B - B_{in}$) follows:

⇨ Difference (D): $D = A \oplus B \oplus B_{in}$

- Borrow-out (B_{out}): $B_{out} = \bar{A}B + (\bar{A} + B)B_{in}$

C. Signed and Unsigned Data Representation

The circuit produces a 4-bit output string that is interpreted in two distinct formats:

- Unsigned Logic:
  - Treats all 4 bits as positive magnitude.
  - Weighting: 2^3, 2^2, 2^1, 2^0
  - Range: [0, 15]
- Signed Logic (2's Complement):
  - The Most Significant Bit (MSB) acts as the sign bit.
  - Weighting: -2^3, 2^2, 2^1, 2^0 (MSB has a negative weight of -8).
  - Range: [-8, +7]
- Conversion Formula: Both the MATLAB and JavaScript engines apply the logic:

  If MSB (S3) == 1, Signed Value = Unsigned Value - 16.

D. Overflow Detection (V)

Signed overflow occurs when the magnitude of the arithmetic result exceeds the capacity of the sign bit, causing a "wrap-around" error.

- Logic: Overflow only applies to signed arithmetic. It indicates that two positive numbers yielded a negative result, or vice versa.
- Implementation: The circuit detects this by comparing the carry entering the MSB (C_3) and the carry exiting the MSB (C_4):

  $V = C_3 \oplus C_4$

- Significance: If V=1, the signed interpretation is mathematically invalid, while the unsigned interpretation remains valid (C_4 handles the extra magnitude).

TEST CASES & RESULTS

The project was validated using a series of strategic test cases designed to verify the accuracy of the ripple carry logic, 2's complement subtraction, and error detection (carry and overflow flags). The following results were consistent across both the Web-Based Simulator and the MATLAB implementation.

A. Addition Test Cases (Mode = 0)

Case	Input A	Input B	Binary Result	Unsigned	Signed	Status
Simple Addition	0011 (3)	0010 (2)	0101	5	5	Correct
Negative + Positive	1001 (-7)	0110 (6)	1111	15	-1	Correct
Signed Overflow	0111 (7)	0010 (2)	1001	9	-7	Overflow Detected

Observation (Case 3): In signed mode, 7 + 2 = 9. Since 9 is outside the representable range [-8, +7] for a 4-bit signed integer, the MSB becomes 1. The circuit correctly identifies this as an error by detecting that the carry into the MSB (C_3) and the carry out (C_4) are different (V = C_3 \oplus C_4 = 1).

B. Subtraction Test Cases (Mode = 1)

Case	Input A	Input B	Binary Result	Unsigned	Signed	Status
Simple Subtraction	0111 (7)	0011 (3)	0100	4	4	Correct
Result < 0 (Borrow)	0011 (3)	0101 (5)	1110	14	-2	Correct
Zero Result	0101 (5)	0101 (5)	0000	0	0	Correct

Observation (Case 2): When A < B in unsigned mode, a "borrow" occurs, which is indicated by Carry Out = 0. However, in signed mode, the result 1110 is the mathematically accurate representation of -2 using the 2's complement system.

C. Summary of Performance Results

- Logical Functional Accuracy: The XOR gates correctly performed conditional inversion of Input B during subtraction. The initial carry-in (C_0 = 1) successfully completed the 2's complement conversion without requiring a separate subtractor circuit.
- Interpretation Consistency: The dual-display logic accurately converted binary bitstreams to decimal in real-time. The formulas used in the JavaScript UI and the MATLAB script yielded identical decimal values for all 256 possible input combinations.
- Flag Verification: The Carry flag correctly identified unsigned range extensions, while the Overflow (V) flag successfully triggered only during signed operations where the result exceeded the 4-bit boundary.

CONCLUSION

A. Conclusion and Key Findings

The Binary Calculator Using Logic Gates project successfully demonstrates the practical application of Boolean algebra and digital electronics in modern computing systems. By implementing a 4-bit Ripple Carry Adder-Subtractor, the project illustrates how a single hardware circuit can be multiplexed to perform multiple arithmetic operations using the 2's Complement method.

The project highlights the following key technical insights:

- Hardware Efficiency: The strategic use of XOR gates for conditional inversion proves that addition and subtraction do not require separate discrete circuits, mirroring the architectural efficiency of real-world Arithmetic Logic Units (ALUs).
- Data Interpretation: The implementation underscores that binary data is inherently neutral; its meaning (Unsigned vs. Signed) depends entirely on the architectural interpretation of the Most Significant Bit (MSB).
- Error Diagnostics: The integration of Carry-out and Overflow flags provides a clear distinction between unsigned range exhaustion and signed arithmetic errors ($V = C_3 \oplus C_4$).
- Cross-Platform Consistency: Developing the logic in both MATLAB and JavaScript verifies that the underlying mathematical algorithms are platform-independent, robust, and mathematically sound across different runtime environments.

B. Learning Outcomes

Through the development of this simulator, the following core competencies were mastered:

- Digital Logic Design: Proficiency in using XOR gates as conditional inverters to streamline circuit complexity and Cascaded Full Adders to manage carry propagation.
- Mathematical Rigor: A deep understanding of 2's complement arithmetic and its critical role in simplifying binary subtraction at the hardware level.
- Circuit Optimization: Expertise in deriving and verifying optimized Boolean expressions using Truth Tables, K-Maps, and gate-level simulation.
- Diagnostic Logic: The ability to implement hardware-level error detection (Carry vs. Overflow) for different numerical representations, ensuring data integrity.

C. Future Scope

While this project focuses on a 4-bit architecture for educational clarity, the logic is inherently scalable. Future enhancements could include:

1. Bit-Width Expansion: Scaling the simulator to 8-bit or 16-bit to demonstrate higher-order arithmetic and increased propagation delay.
2. Look-Ahead Carry Unit: Implementing a Carry Look-Ahead (CLA) unit to demonstrate how to reduce the time complexity of ripple carry addition.
3. Extended ALU Functions: Integrating bitwise logic operations (AND, OR, NOT, NAND) to transform the calculator into a more comprehensive ALU simulator.

LINK OF THE WORKING PROJECT

The complete source code and interactive elements of this project can be accessed through the following files:

- Interactive Web Simulator: index.html (Open in any modern web browser)
- MATLAB Algorithm: binary_calculator.m (Run in MATLAB R2016b or later)
- Project Documentation: README.md

LINK:

https://github.com/jai2004raj/Binary-Calculator-Using-Logic-gates